

シンプル計算機アプリ 解説資料

制作の流れ・ファイルの関係・各ファイルの役割・React の使い方

対象リポジトリ: simple-calculator 技術: Vite + React 19 + TypeScript + Tailwind CSS v4

作成日: 2026-09-12

先に結論: このアプリは **Next.js** ではありません。ブラウザで動く1ページの React アプリを、**Vite** という開発サーバー兼ビルドツールで動かしています。Next.js の `app/page.tsx`、サーバーコンポーネント、ルーティングは使っていません。

目次

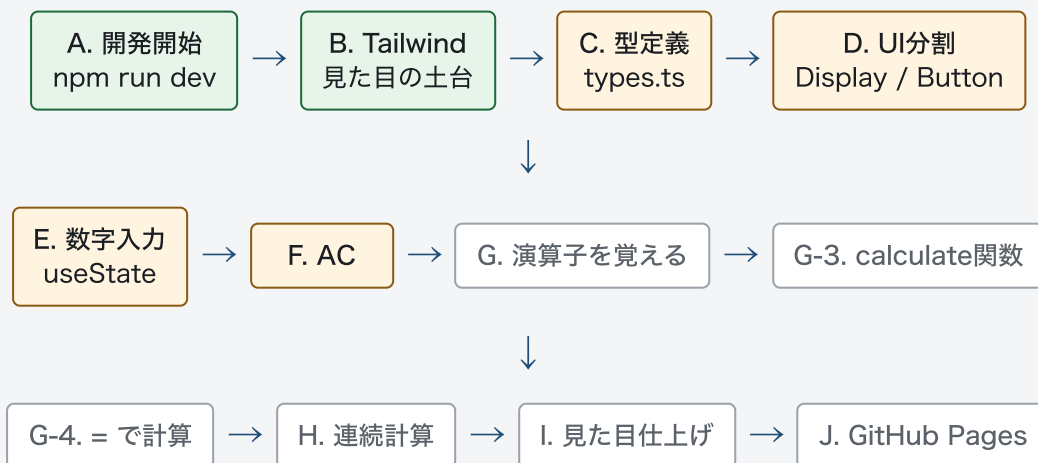
1. 制作の流れ
2. ファイルの関係性
3. 各ファイルが何をしているか
4. React / Vite の機能の使い方 (Next.js との違い含む)

1. 制作の流れ

1-1. 学習課題としての全体像

目標は「iPhone 風の電卓」を、現場に近い進め方で作ることです。見た目 → 数字入力 → 計算 → 連続計算 → 公開、の順に小さく進めます。

図1. 制作ロードマップ (README の ToDo に対応)



□ 完了に近い □ 途中（いまここ） □ これから

出典: リポジトリ README の「自分で進める細かいタスク」と、現在の src コード。

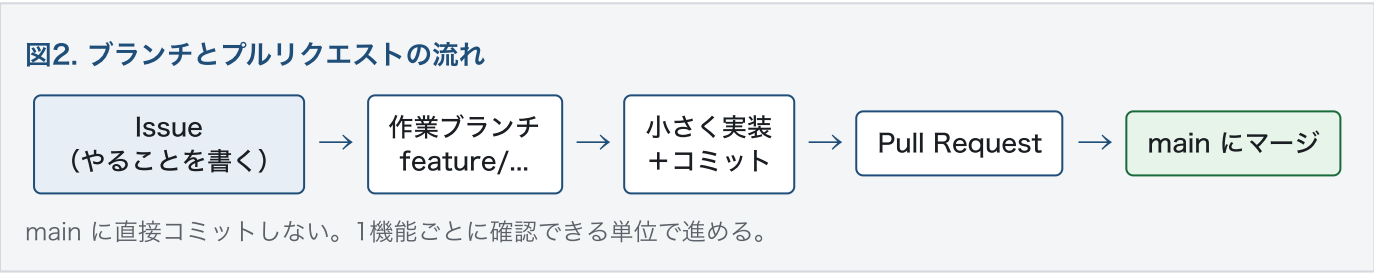
1-2. いまの実装位置

骨格はあります。計算ロジックはまだ途中です。

できていること	まだ途中のこと
Vite + React + TypeScript の起動 Tailwind の className で電卓の枠を描画 Display / Button への分割 数字ボタンで画面の数字が変わる AC で表示が 0 に戻る	数字の連結（今は押した1桁で上書き） 小数点入力 +-x÷ の計算 連続計算 types.ts を App から使う iPhone 風の色分け・0ボタン横長 GitHub Pages 公開

1-3. Git / GitHub の進め方（現場想定）

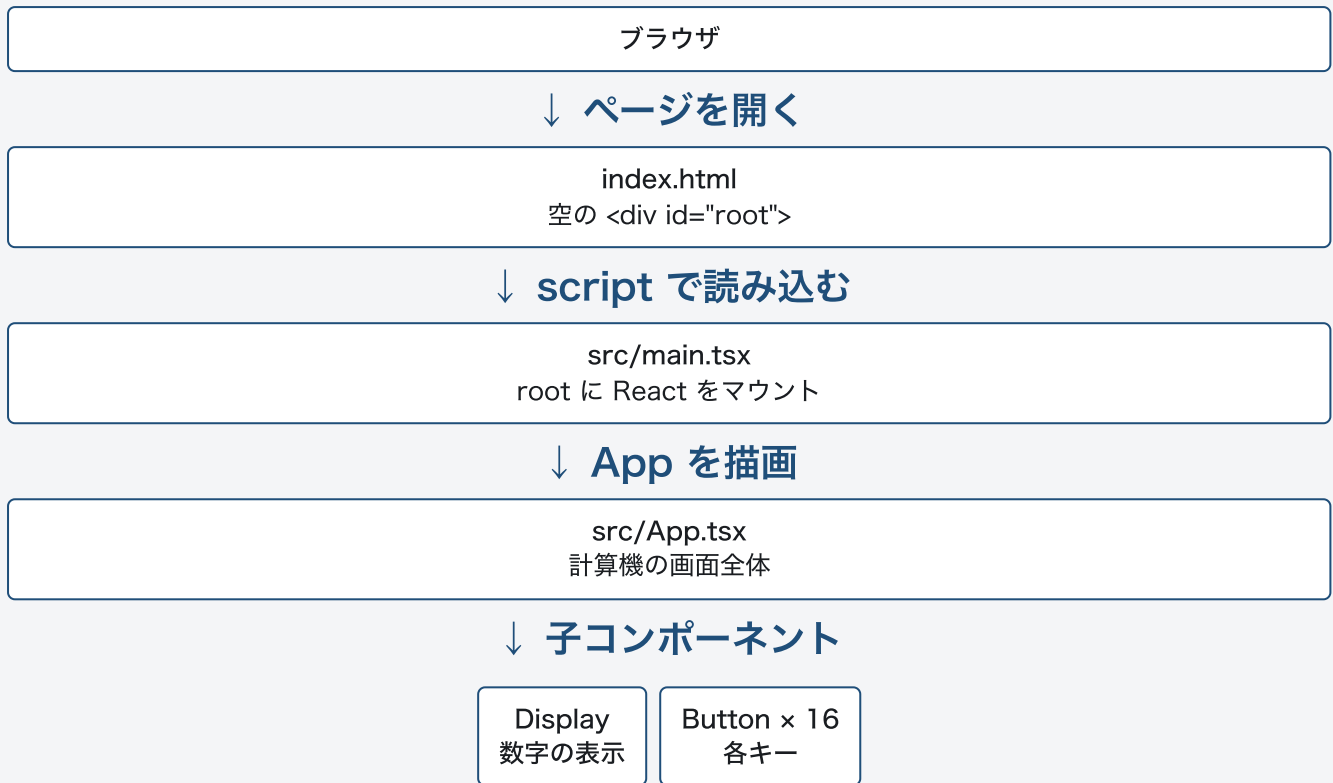
コードの作り方とは別に、提出用の流れは次のとおりです。



1-4. パソコンを起動して画面が出るまでの流れ

`npm run dev` を実行すると、Vite が開発サーバーを立て、ブラウザが HTML を読みます。

図3. 起動時の読み込み順



1-5. ボタンを押したあとのデータの流れ

いま動いているのは「数字を押す → 表示をその数字にする」です。計算はまだしません。

図4. 数字ボタンをクリックしたときの流れ（現在の実装）



次に足す流れ（まだ未実装）： 数字は末尾に足す → 演算子を覚える（previousValue / operator） → 次の数字を入力 → = で calculate → 結果を表示。連続計算では、演算子を押した時点で先に計算します。

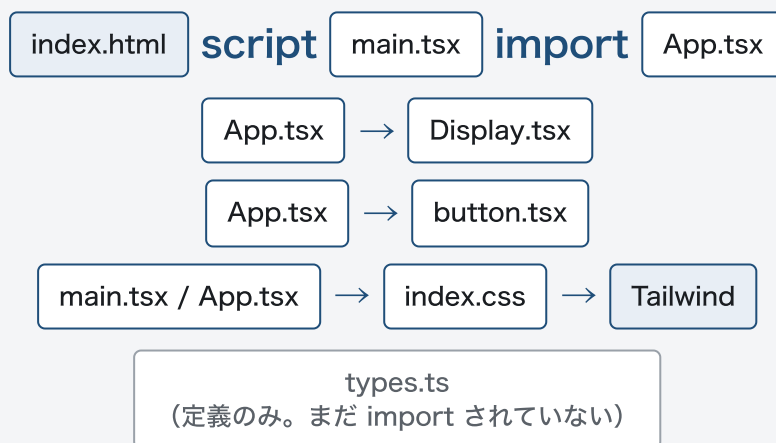
2. ファイルの関係性

アプリ本体は `src/` にあります。設定ファイルはルートに置き、実行時の画面には出ません。

図5. フォルダ構成（実行に関係するもの）

simple-calculator/	
├ index.html	← 最初に読まれる HTML
├ package.json	← 使うライブラリと npm スクリプト
├ vite.config.ts	← Vite と Tailwind / React プラグイン
├ tsconfig*.json	← TypeScript の設定
├ public/	← favicon など、そのまま公開する静的ファイル
└ src/	
├ main.tsx	← React の入口
├ index.css	← Tailwind の読み込み
├ App.tsx	← 計算機の親コンポーネント
├ App.css	← 今は空（Vite 初期の名残）
├ types.ts	← 演算子などの型（まだ他から未使用）
└ components/	
├ Display.tsx	← 画面の数字表示
└ button.tsx	← 1つのキー

図6. import の関係（矢印は「読み込んでいる」方向）



Header はファイルではなく App.tsx の中に書いた小さなコンポーネントです。

実行時に画面を作る層

index.html → main.tsx → App.tsx →
Display / Button

開発を支える層

package.json、vite.config.ts、
tsconfig、eslint.config.js

Props でつながる親子関係

図7. 親 App から子へ渡している値			
親	子	渡している Props	意味
App	Header	title, className	見出し「計算機」と見た目
App	Display	value, className	今の数字を表示する
App	Button	label, className, onClick	キーの文字・見た目・押したときの処理

子は自分で計算しません。何を表示し、押されたら親のどの関数を呼ぶか、親が決めます。

3. 各ファイルが何をしているか

3-1. アプリ本体

ファイル	役割	いまやっていること
index.html	ページの器	タイトル、言語、favicon を設定。 <code><div id="root"></code> は空で、React が中身を描き込む。 <code>/src/main.tsx</code> をモジュールとして読み込む。
src/main.tsx	React の入口	<code>createRoot</code> で root にアプリを載せる。 <code>StrictMode</code> で開発中のチェックを有効にする。 <code>App</code> と全体 CSS を読み込む。
src/App.tsx	計算機の本体（親）	<code>useState("0")</code> で表示値を持つ。 <code>Header</code> / <code>Display</code> / 16 個の <code>Button</code> を並べる。数字と AC のクリックで state を更新する。Tailwind で枠・グリッド・余白を指定する。
src/components/Display.tsx	ディスプレイ	<code>value</code> （文字列）を受け取って表示するだけ。計算はしない。表示専用の部品。
src/components/button.tsx	キー1個	<code>label</code> をボタン文字にする。 <code>onClick</code> があれば押したときに実行。演算子ボタンの一部は <code>onClick</code> 未設定。
src/types.ts	型の置き場	<code>Operator</code> <code>Digit</code> などは定義済み。 <code>CalculatorState</code> などは空。他ファイルから <code>import</code> されていない。
src/index.css	全体スタイルの入口	<code>@import "tailwindcss";</code> のみ。これで utility class が使える。
src/App.css	（未使用）	中身は空。Vite テンプレートの名残。

3-2. 設定・開発用ファイル

ファイル	役割
package.json	プロジェクト名、依存関係、スクリプト。 <code>npm run dev</code> は Vite 起動、 <code>npm run build</code> は TypeScript チェック後に本番用ファイルを作る。
vite.config.ts	Vite の設定。 <code>@vitejs/plugin-react</code> で JSX を変換。 <code>@tailwindcss/vite</code> で Tailwind をビルドにさせる。
tsconfig.json	TypeScript 設定の親。アプリ用と Vite 用に分割して参照する。
tsconfig.app.json	src 向け。JSX は <code>react-jsx</code> (import React 不要)。ブラウザ向けの型 (DOM) を使う。
tsconfig.node.json	<code>vite.config.ts</code> 向け。Node 上で読む設定ファイル用。
eslint.config.js	コードの書き方チェック。React Hooks のルールも含む。
public/	ビルドしても中身を変換しない静的ファイル (favicon など)。
README.md	課題説明と、A〜K の学習タスク一覧。

3-3. 名前の注意 (Button の import)

`App.tsx` は `./components/Button` を読み、実ファイル名は `button.tsx` です。macOS の標準ディスクは大文字小文字を区別しないことが多いので動きます。GitHub の Linux 環境では失敗することがあるので、ファイル名と import を揃えるのが安全です。

4. React / Vite の機能の使い方

Next.js 特有の機能 (App Router、サーバーコンポーネント、`next/link`、SSR) は使っていません。使っているのは **React 本体** と **Vite** です。

4-1. このプロジェクトで使っている React

機能	どこで	このアプリでの意味
関数コンポーネント	App / Header / Display / Button	「画面の部品 = 関数」。受け取った値を JSX で返す。クラスコンポーネントは使っていない。
JSX	各 .tsx	HTML に似た記法で UI を書く。属性は <code>className</code> 、 <code>onClick</code> のように React 用の名前。
Props	Display / Button / Header	親から子へデータを渡す。Display は <code>value</code> 、Button は <code>label</code> と <code>onClick</code> 。

機能	どこで	このアプリでの意味
useState	App.tsx	画面に出している文字列を覚える。 <code>const [displayValue, setDisplayValue] = useState("0")</code> 。更新すると App 以下が再描画される。
イベントハンドラ	Button の onClick	クリックで親の setter を呼ぶ。例: <code>onClick={() => setDisplayValue("8")}</code>
コンポーネント分割	components/	表示とボタンを再利用可能な部品にする。同じ Button を 16 回使い、label だけ変える。
createRoot	main.tsx	React 19 のマウント方法。HTML の root に仮想 DOM の木を載せる。
StrictMode	main.tsx	開発中だけ、副作用の間違いを見つけやすくする。本番の見たい目は変えない。
名前付き export / default export	各ファイル	App は default export。Display / Button は <code>export function</code> （名前付き）。読み込み方が違う。
再レンダリング	state 更新時	setDisplayValue すると App が再実行され、新しい value が Display に渡る。手動で DOM を書き換えない。

```
// App.tsx の中心（状態は親が持つ）
const [displayValue, setDisplayValue] = useState("0");

<Display value={displayValue} />
<Button label="8" onClick={() => setDisplayValue("8")} />
<Button label="AC" onClick={() => setDisplayValue("0")} />
```

4-2. TypeScript の使い方

- Props の型（ `ButtonProps` 、 `DisplayProps` ）で、渡し忘れをコンパイル時に防ぐ。
- `className?` と `onClick?` は任意。演算子ボタンはまだ `onClick` なしでも型エラーにならない。
- `types.ts` の `Operator = "+" | "-" | "x" | "÷"` はユニオン型。許す文字だけを型で固定する予定。
- 表示は `string`、計算するときだけ `number` にする、という方針が Display のコメントに書いてある。

4-3. Vite の使い方（Next.js の代わりにになっている部分）

やりたいこと	Next.js なら	このプロジェクトでは
開発サーバー	<code>next dev</code>	<code>npm run dev</code> → Vite
ページの入口	<code>app/page.tsx</code>	<code>index.html</code> + <code>src/main.tsx</code>
JSX の変換	Next が内蔵	<code>@vitejs/plugin-react</code>
保存したらすぐ反映	Fast Refresh	Vite の HMR + React プラグイン
本番用ファイル	<code>next build</code>	<code>npm run build</code> （tsc のあと vite build）
複数ページ / SSR	標準機能	使わない。電卓は1画面の SPA

4-4. Tailwind CSS の使い方

- `vite.config.ts` に `tailwindcss()` プラグイン。
- `index.css` で `@import "tailwindcss"`。
- コンポーネントの `className` に utility を書く。

className の例	効果
<code>bg-zinc-900 rounded-lg w-[80%] mx-auto p-20</code>	暗い背景、角丸、幅80%、中央、大きな余白
<code>grid grid-cols-4 gap-4</code>	ボタンを4列の格子に並べる
<code>text-white text-4xl font-bold</code>	白い大きな太字
<code>text-center font-bold mt-15 mb-6</code>	見出しを中央・太字・上下余白

4-5. データと描画の役割分担（覚えると計算実装が楽）

図8. これから計算を足すときの役割分担

層	担当ファイル（予定）	仕事
表示	Display.tsx	文字列を出すだけ
入力	button.tsx	押されたことを親に伝えるだけ
状態	App.tsx（または Calculator.tsx）	今の数字、前の数字、演算子を useState で持つ
計算	utils/calculate.ts（未作成）	UI を知らない純粋な関数。例: calculate(1, 2, "+") → 3
型	types.ts	許す演算子・状態の形を固定する

4-6. Next.js を後から使うとしたら

今の電卓に Next.js は必須ではありません。もし乗せるなら、だいたい次の対応になります。

- `src/App.tsx` の中身 → `app/page.tsx` (クライアントで state を使うなら先頭に `"use client"`)
- `src/main.tsx` と `index.html` → Next.js が用意するので不要になる
- `Display / Button` はそのままコンポーネントとして使える
- ページが1つなので、ルーティングの恩恵はほぼない

まとめ

1. **制作の流れ**は、環境 → 見た目 → 型 → 部品分割 → state で数字 → 計算関数 → 公開。
2. **ファイル関係**は `index.html` → `main.tsx` → `App.tsx` → `Display / Button`。
3. **App が状態を持ち**、子は表示とクリック伝達に専念している。
4. **React** のコンポーネント・Props・useState・onClick を使っている。**Next.js は未使用**で、開発基盤は Vite。

この資料はリポジトリの現行コード (`App.tsx` / `Display.tsx` / `button.tsx` / `main.tsx` / `types.ts` / `README.md`) に基づく。ブラウザで HTML を開き、印刷 → PDF 保存でも再出力できる。